Bala-Join: An Adaptive Hash Join for Balancing Communication and Computation in Geo-Distributed SQL Databases

(42nd IEEE International Conference on Data Engineering, May 2026)

Wenlong Song[1], Hui Li[1*], Bingying Zhai[1], Jinxin Yang[1], Pinghui Wang[2], Luming Sun[3], Ming Li[3], Jiangtao Cui[1]

[1]Xidian University
[2] Xi'an Jiaotong University
[3]Inspur DataBase Company Ltd.

Contact US: hli@xidian.edu.cn

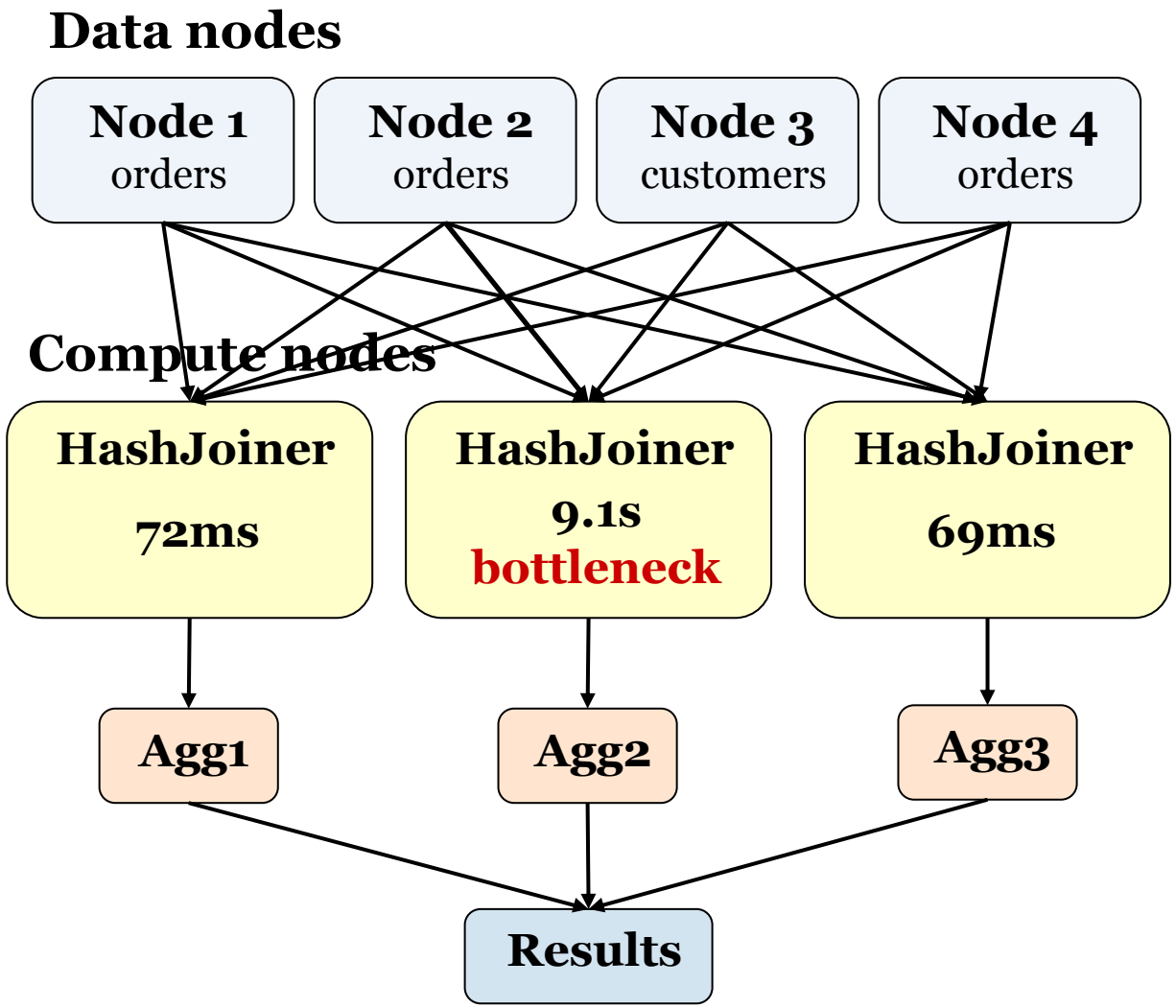
Outline

- Introduction
- Preliminaries
- Our schema
- Conclusions

Introduction: Distributed Hash-Join

- Shared-nothing architecture
  - No shared storage or memory
  - Data partitioning for high scalability
- Distributed Hash-Join (Dist-HJ)
  - Tuples are redistributed by join key across nodes
  - **Data** skew causes **network** and **computation** bottlenecks
- Example: TPC-H join with 10% **data skew**

```
SELECT COUNT(*)
FROM CUSTOMER JOIN ORDERS
ON O_CUSTKEY = C_CUSTKEY;
```



Introduction: Our contributions

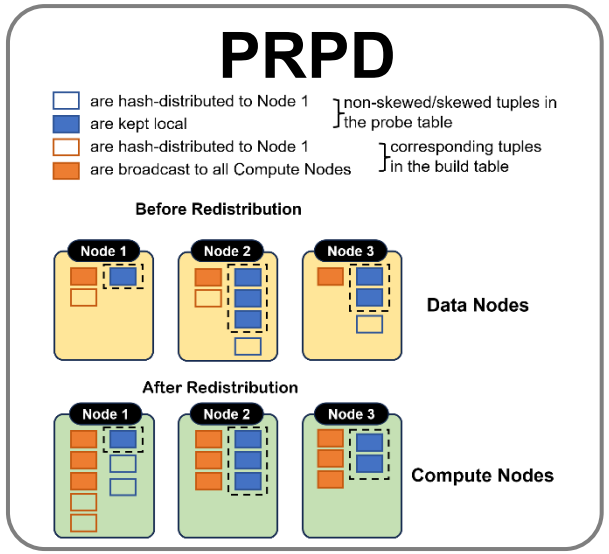
- Contribution 1
  - A new skew-aware redistribution strategy (BPPR) with balanced computation and reduced communication
- Contribution 2
  - An online distributed skew detector without relying on pre-collected statistics
- Contribution 3
  - An efficient end-to-end design (with ASAP) that improves throughput by 25%–61%

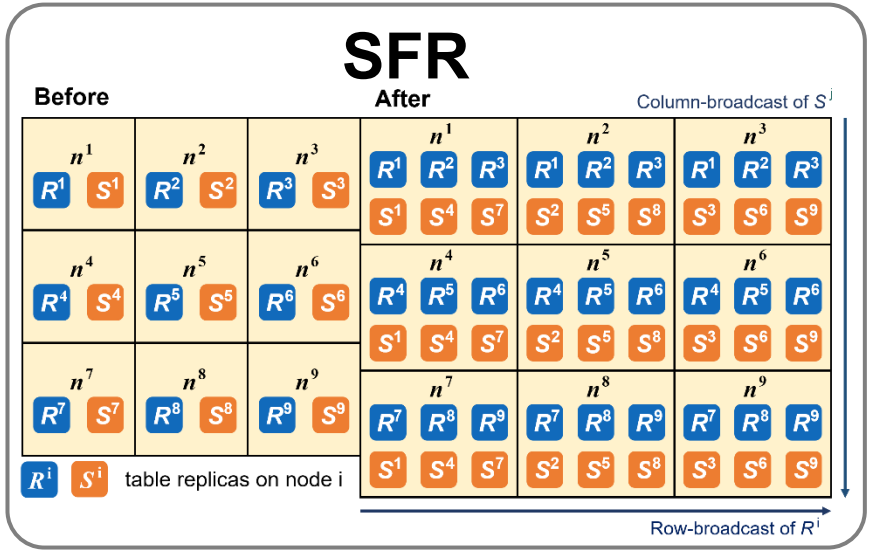
Outline

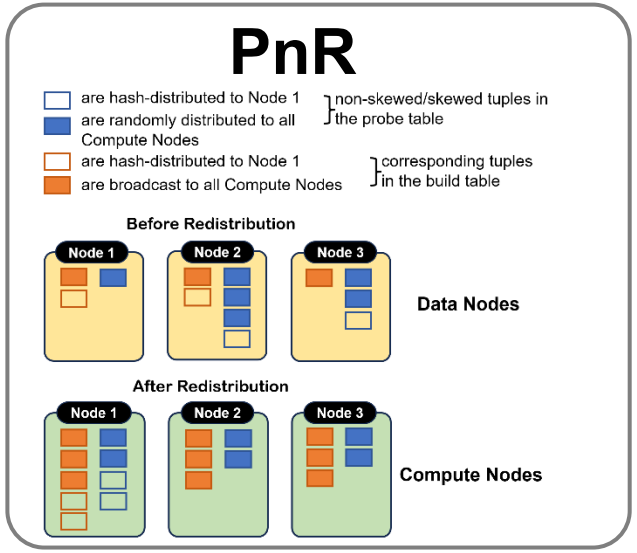
- Introduction
- Preliminaries
- Our schema
- Conclusions

Preliminaries: Dist-HJ strategies

- Existing strategies make different trade-offs, but each has clear limitations.







Good for	Evenly distributed skewed tuples	Severe bilateral skew with sufficient bandwidth	Load balance when skew statistics are known
Not Good for	Concentrated hot keys	Bandwidth-constrained WANs	Streaming / intermediate results or evolving skew
Limitation	Depends on original distribution	High replication cost and possible residual imbalance	Needs skew stats, incurs high network overhead

Preliminaries: Observations and targets

- Observation 1: methods that keep skewed tuples local are extremely sensitive to the original tuple distribution.
- Observation 2: some methods require global statistics of skewed tuples, which are unavailable for streams or intermediate JOIN operators.
- Observation 3: non-adaptive strategies cannot react well when data distributions evolve during execution.

Our target is a solution that is distribution-agnostic, online, and adaptive.

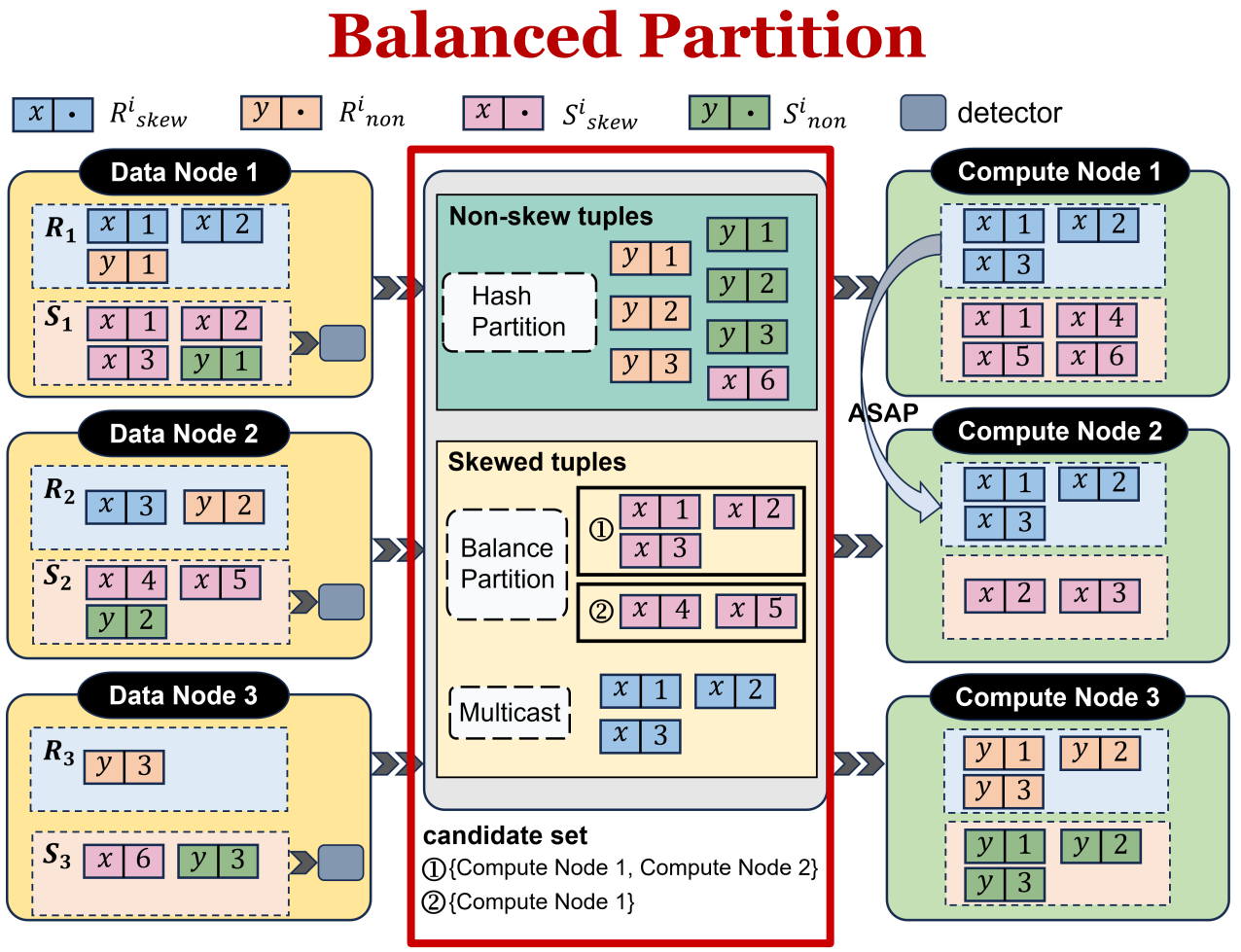
Outline

- Introduction
- Preliminaries
- Our schema: Bala-Join
- Conclusions

Our schema: System Overview

Bala-Join

- **BPPR** for skew-aware redistribution

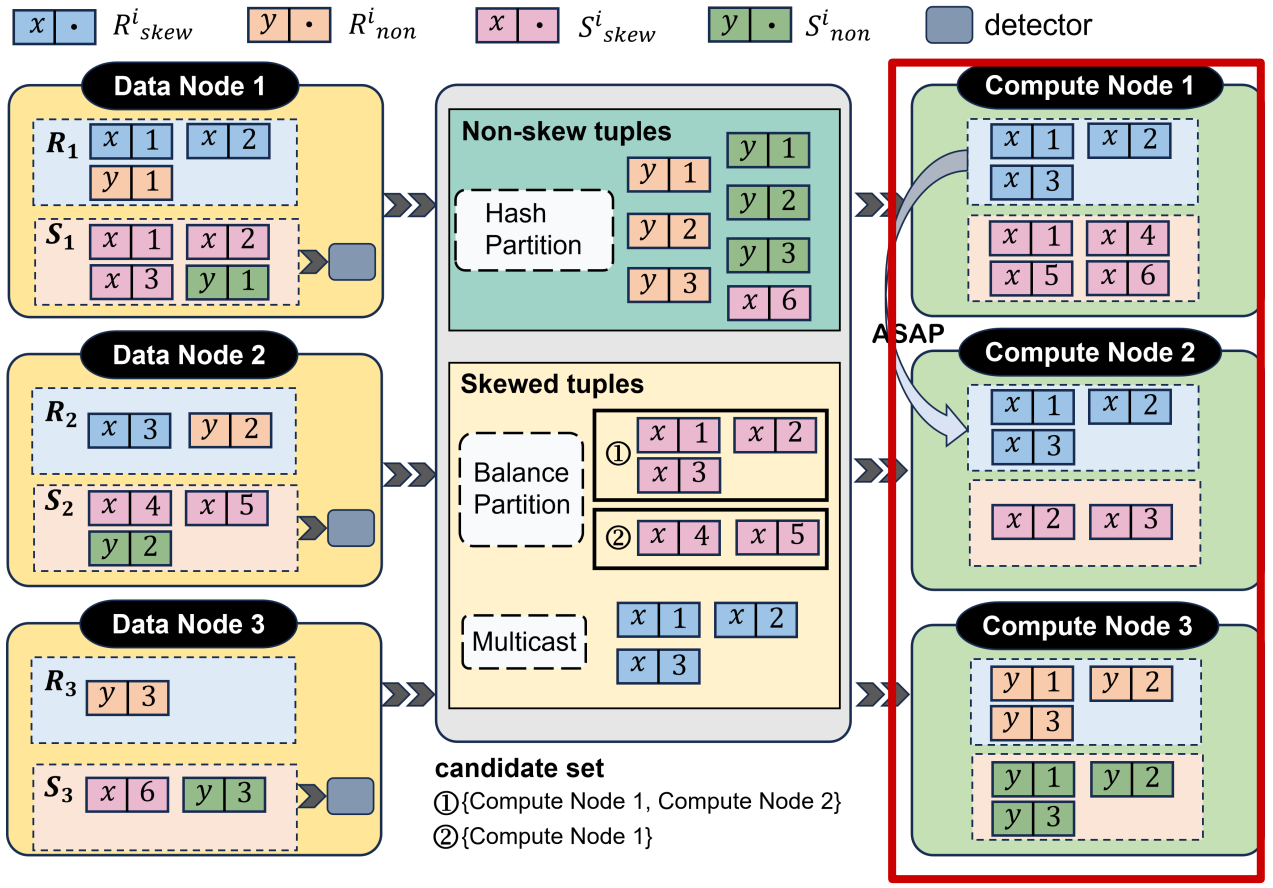


Our schema: System Overview

Bala-Join

- **BPPR** for skew-aware redistribution

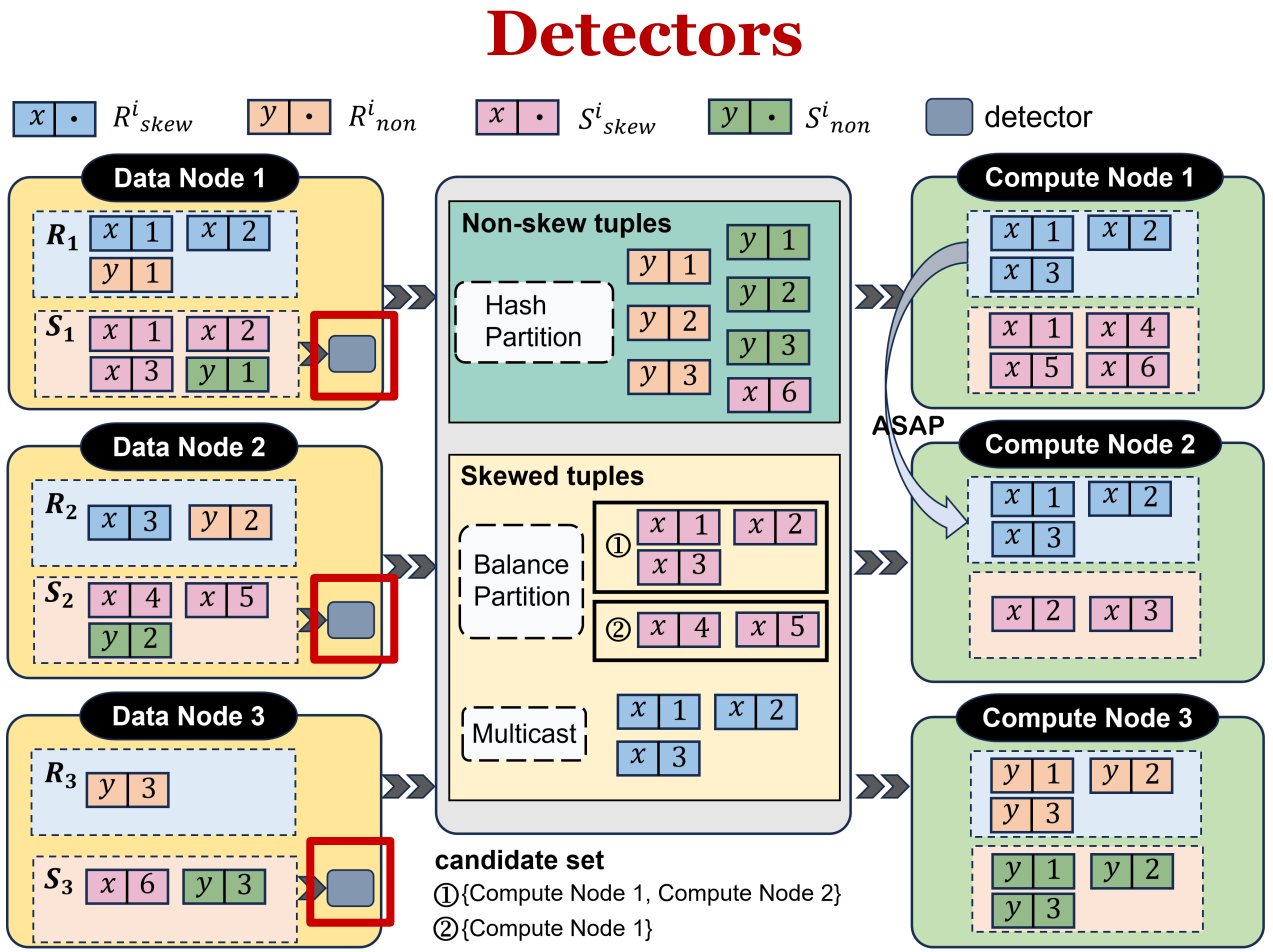
Partial Replication



Our schema: System Overview

Bala-Join

- **BPPR** for skew-aware redistribution
- **A distributed skew detector** for online skew identification

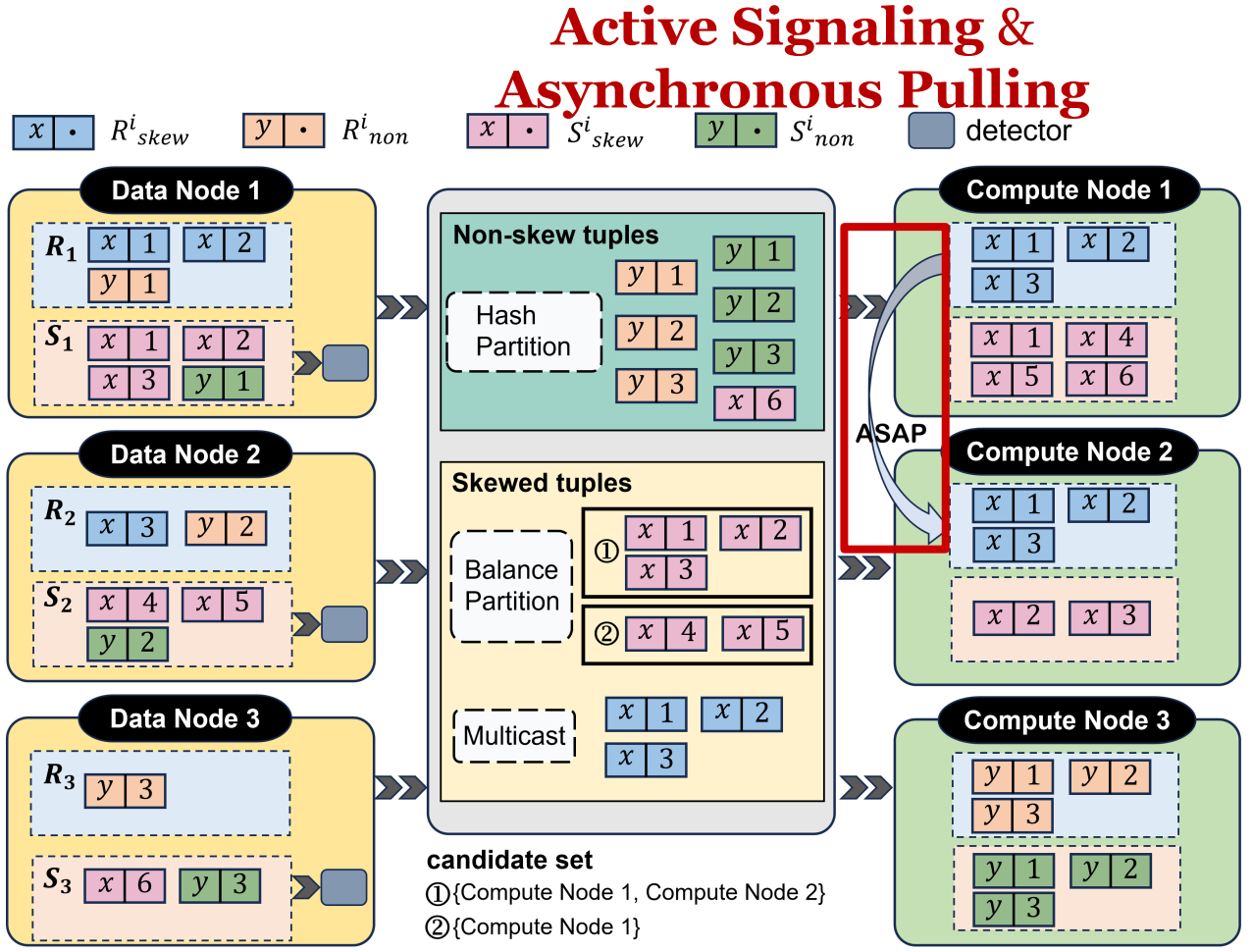


Our schema: System Overview

Bala-Join

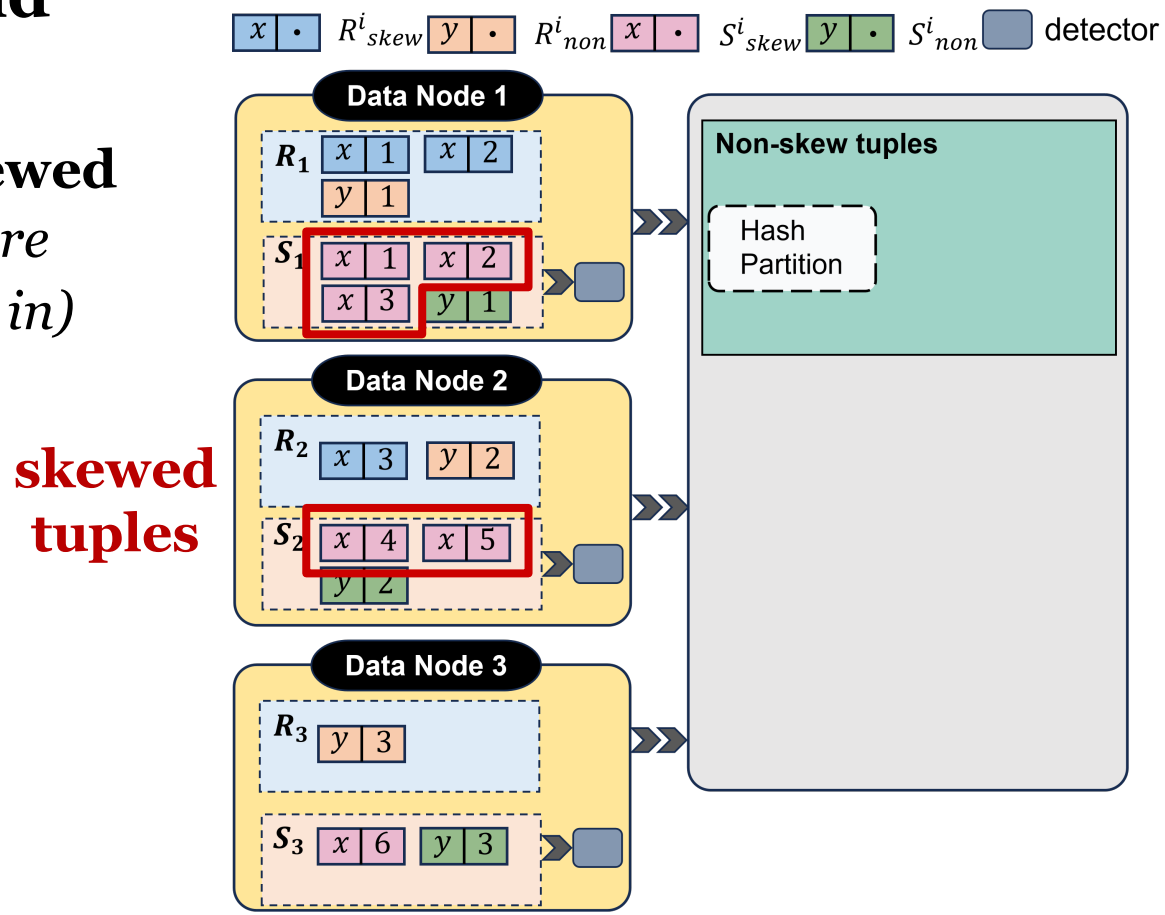
- **BPPR** for skew-aware redistribution
- **A distributed skew detector** for online skew identification
- **ASAP** for efficient build-side synchronization

Core idea: skewed probe tuples are redistributed adaptively, and matching build tuples are synchronized only when needed.



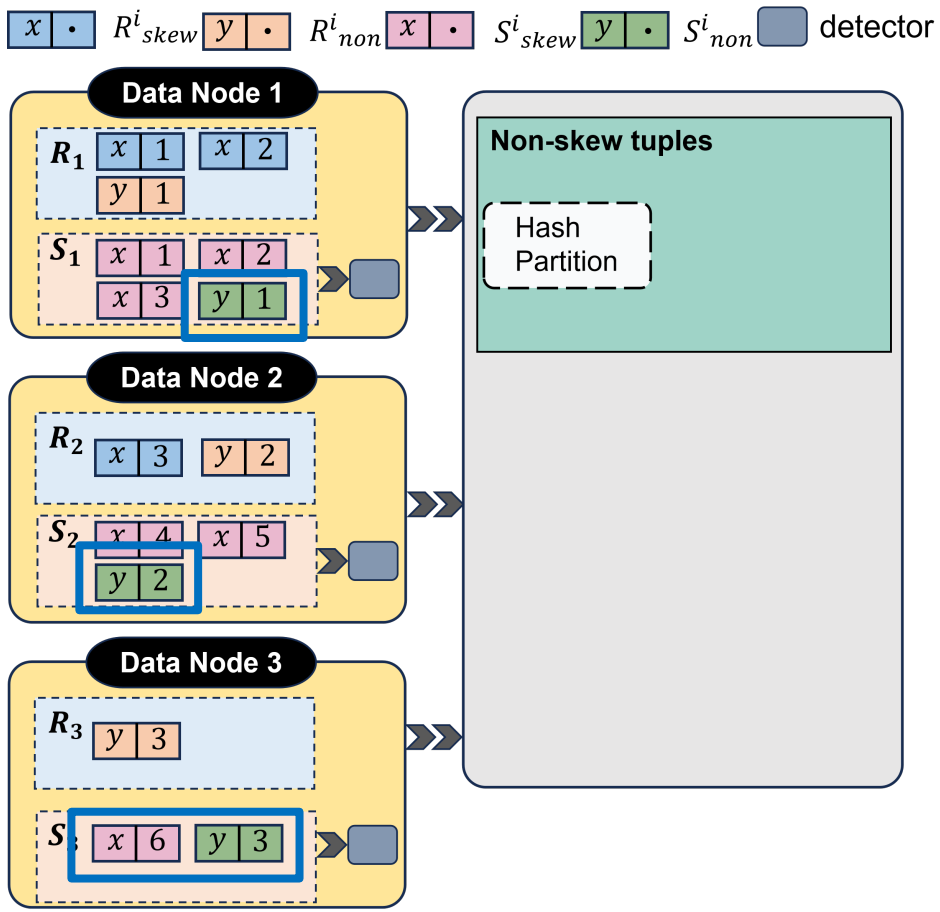
Our schema: Bala-Join

- **BPPR(Balanced Partition and Partial Replication)**
  - Divide the probe side into **non-skewed** and **skewed** tuples *(skewed keys are **detected online** as tuples stream in)*



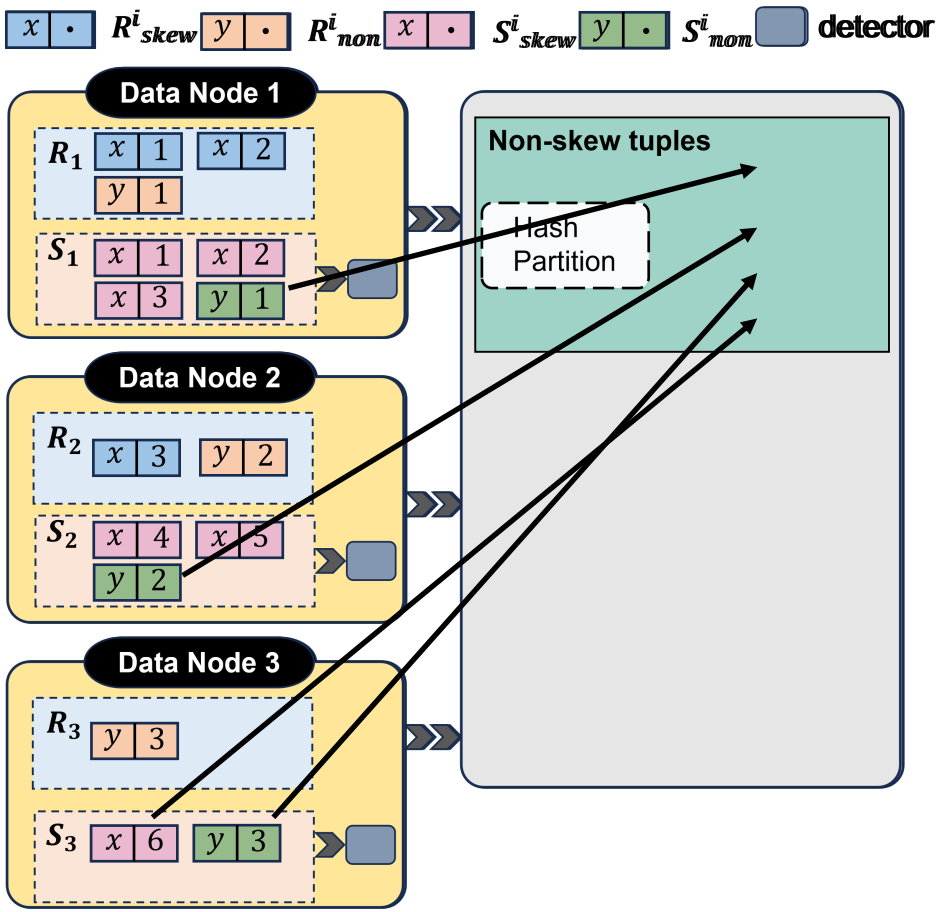
Our schema: Bala-Join

- **BPPR(Balanced Partition and Partial Replication)**
  - Divide the probe side into **non-skewed** and **skewed** tuples *(skewed keys are **detected online** as tuples stream in)*



Our schema: Bala-Join

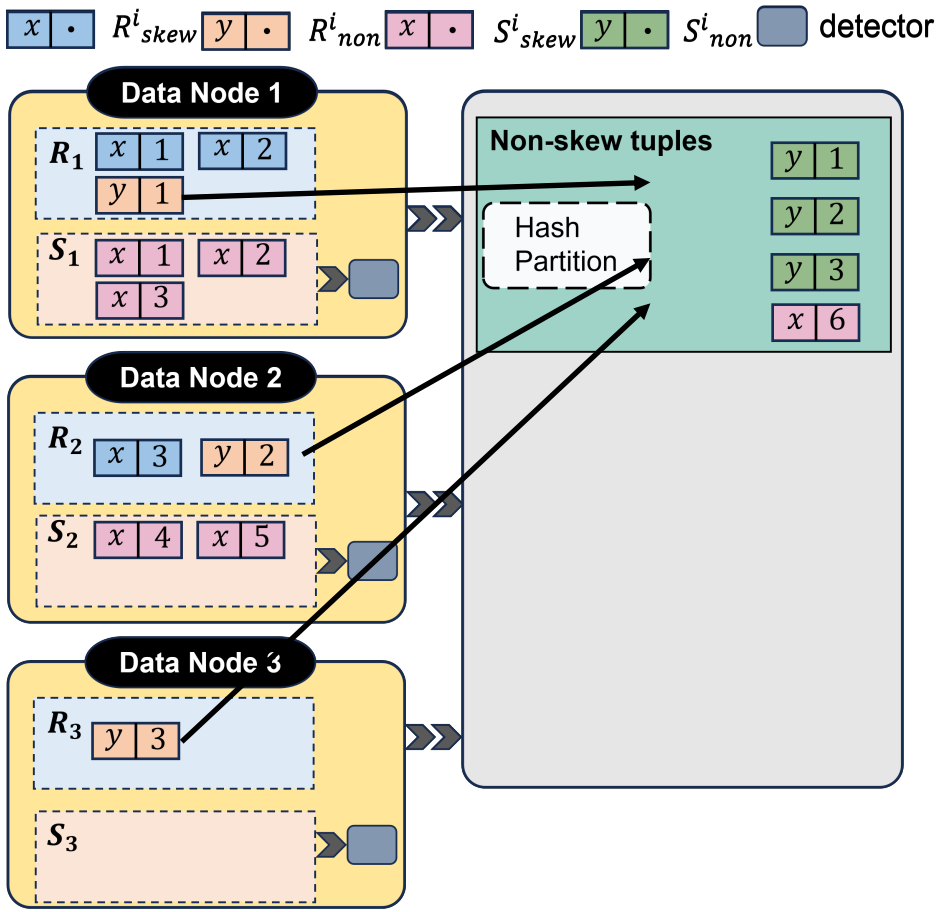
- **BPPR(Balanced Partition and Partial Replication)**
  - Divide the probe side into **non-skewed** and **skewed** tuples *(skewed keys are **detected online** as tuples stream in)*
  - Non-skewed tuples follow **standard hash partitioning**



Logical partitioning

Our schema: Bala-Join

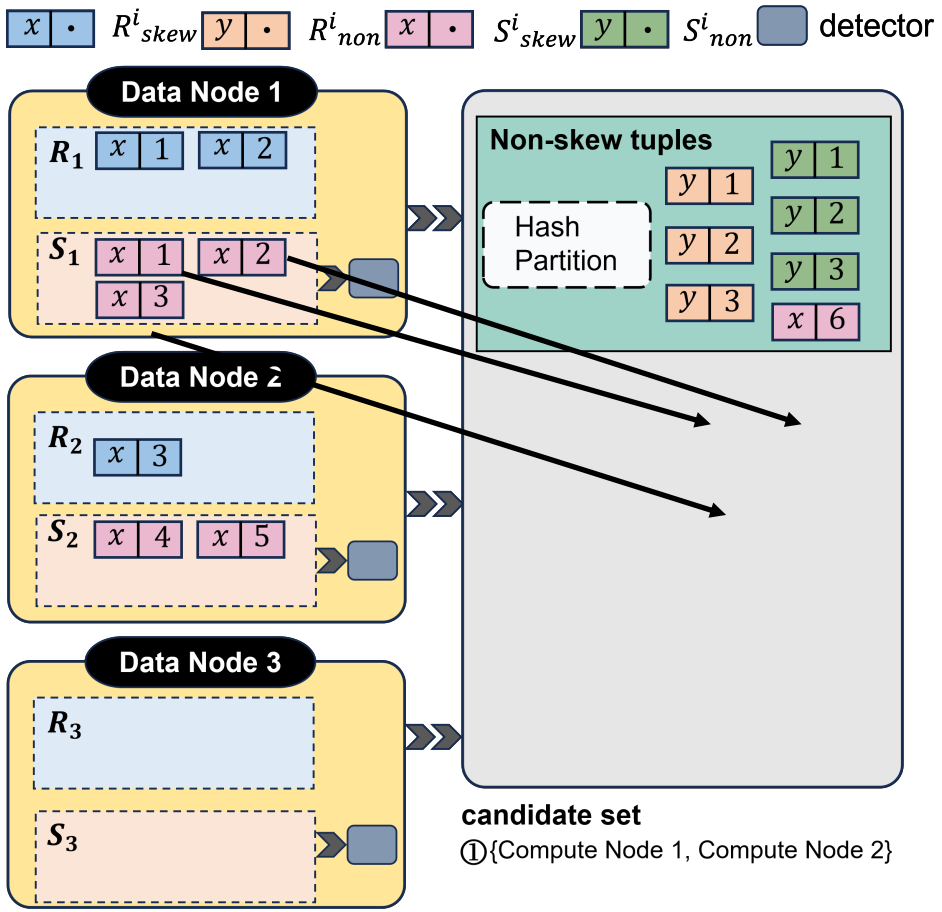
- **BPPR(Balanced Partition and Partial Replication)**
  - Divide the probe side into **non-skewed** and **skewed** tuples *(skewed keys are **detected online** as tuples stream in)*
  - Non-skewed tuples follow **standard hash partitioning**
  - Their matching build-side tuples are also **hash-partitioned**



Logical partitioning

Our schema: Bala-Join

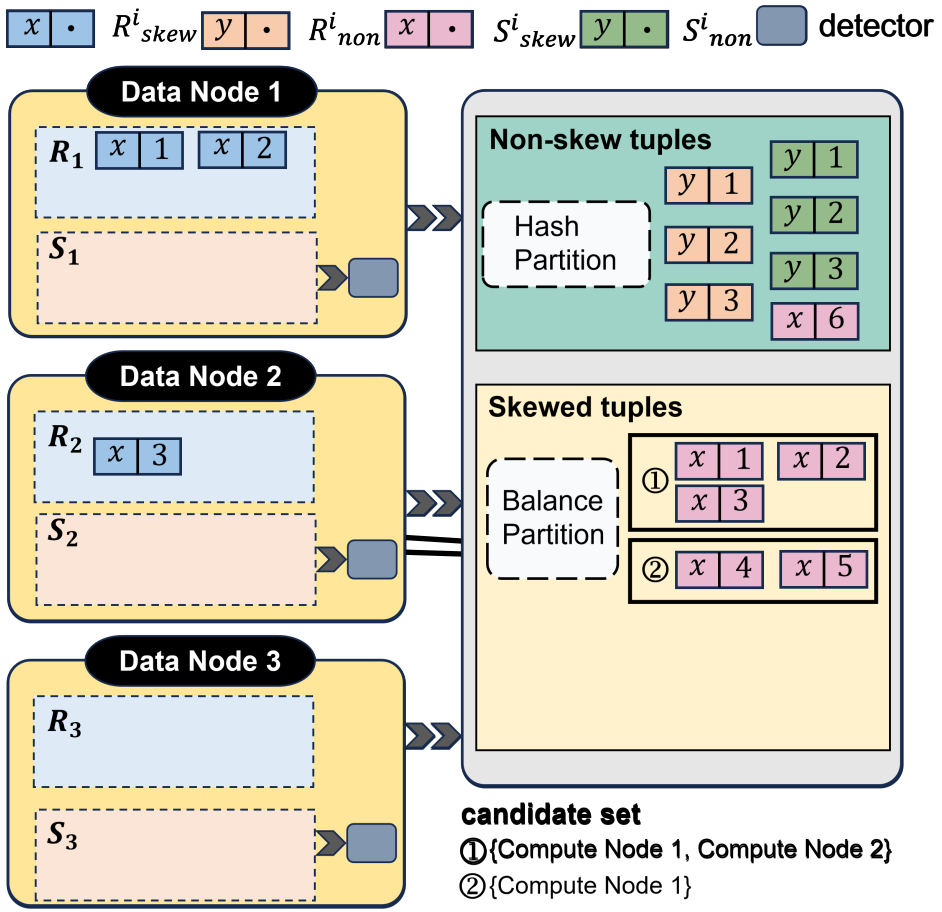
- **BPPR(Balanced Partition and Partial Replication)**
  - Divide the probe side into **non-skewed** and **skewed** tuples *(skewed keys are **detected online** as tuples stream in)*
  - Non-skewed tuples follow **standard hash partitioning**
  - Their matching build-side tuples are also **hash-partitioned**
  - Skewed tuples are routed to a **candidate node set** $U(x)$



Logical partitioning

Our schema: Bala-Join

- **BPPR(Balanced Partition and Partial Replication)**
  - Divide the probe side into **non-skewed** and **skewed** tuples *(skewed keys are **detected online** as tuples stream in)*
  - Non-skewed tuples follow **standard hash partitioning**
  - Their matching build-side tuples are also **hash-partitioned**
  - Skewed tuples are routed to a **candidate node set** $U(x)$

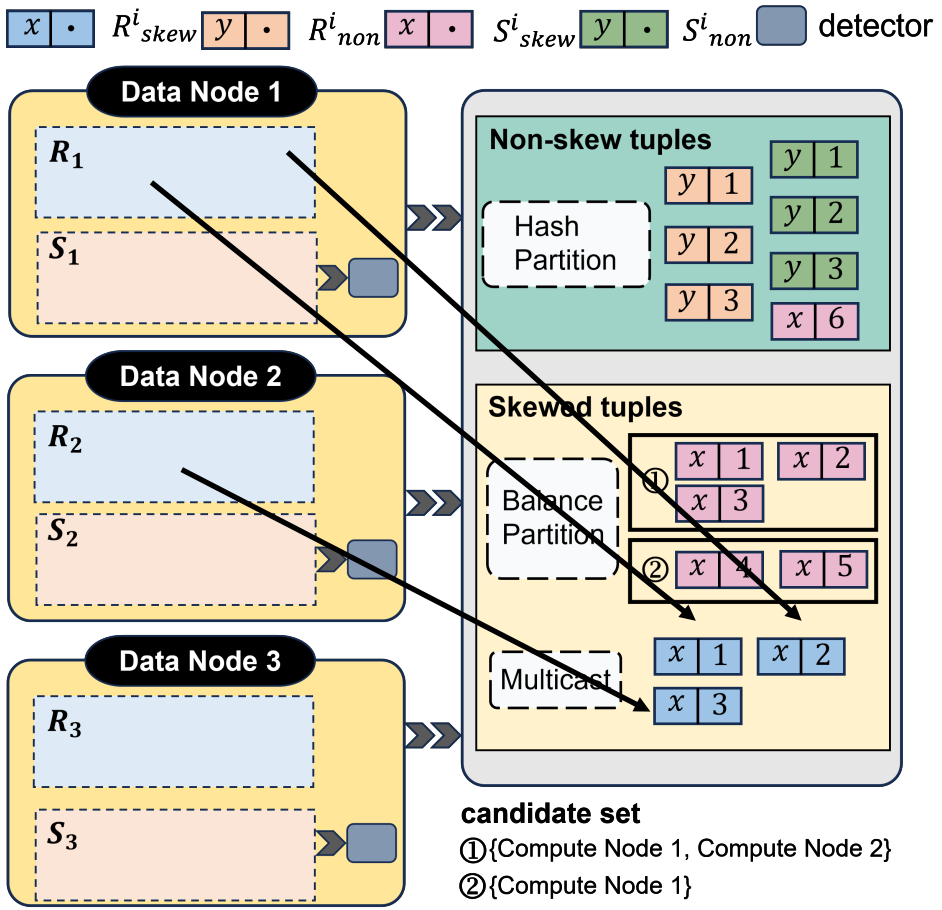


Logical partitioning

Our schema: Bala-Join

BPPR(Balanced Partition and Partial Replication)

- Divide the probe side into **non-skewed** and **skewed** tuples *(skewed keys are **detected online** as tuples stream in)*
- Non-skewed tuples follow **standard hash partitioning**
- Their matching build-side tuples are also **hash-partitioned**
- Skewed tuples are routed to a **candidate node set** $U(x)$
- The matching build-side tuples are then multicast **only to** $U(x)$, reducing unnecessary network overhead

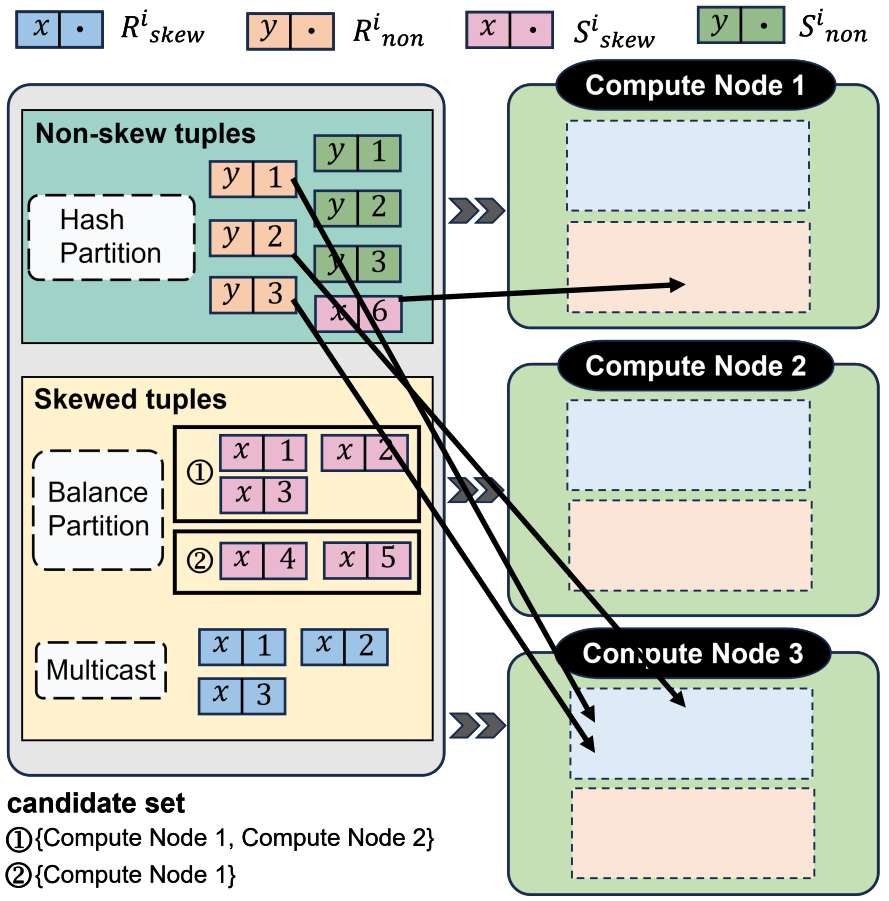


Logical partitioning

Our schema: Bala-Join

BPPR(Balanced Partition and Partial Replication)

- Divide the probe side into **non-skewed** and **skewed** tuples *(skewed keys are **detected online** as tuples stream in)*
- Non-skewed tuples follow **standard hash partitioning**
- Their matching build-side tuples are also **hash-partitioned**
- Skewed tuples are routed to a **candidate node set** $U(x)$
- The matching build-side tuples are then multicast **only to** $U(x)$, reducing unnecessary network overhead

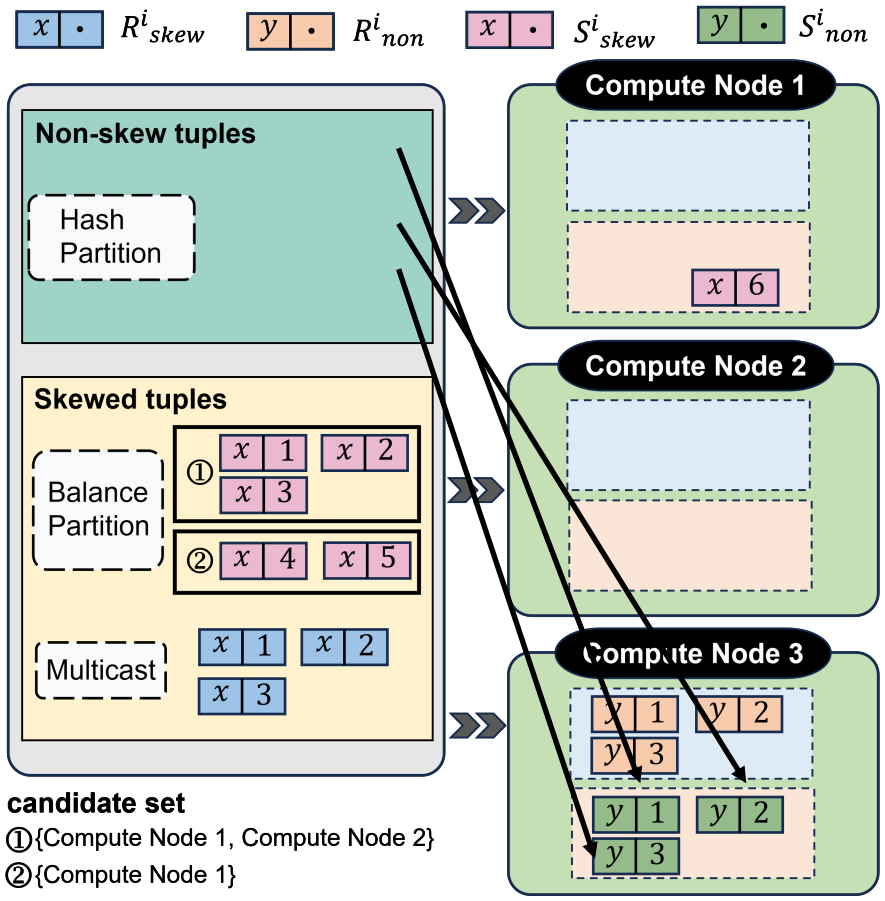


Redistributing partitions to nodes

Our schema: Bala-Join

BPPR(Balanced Partition and Partial Replication)

- Divide the probe side into **non-skewed** and **skewed** tuples *(skewed keys are **detected online** as tuples stream in)*
- Non-skewed tuples follow **standard hash partitioning**
- Their matching build-side tuples are also **hash-partitioned**
- Skewed tuples are routed to a **candidate node set** $U(x)$
- The matching build-side tuples are then multicast **only to** $U(x)$, reducing unnecessary network overhead

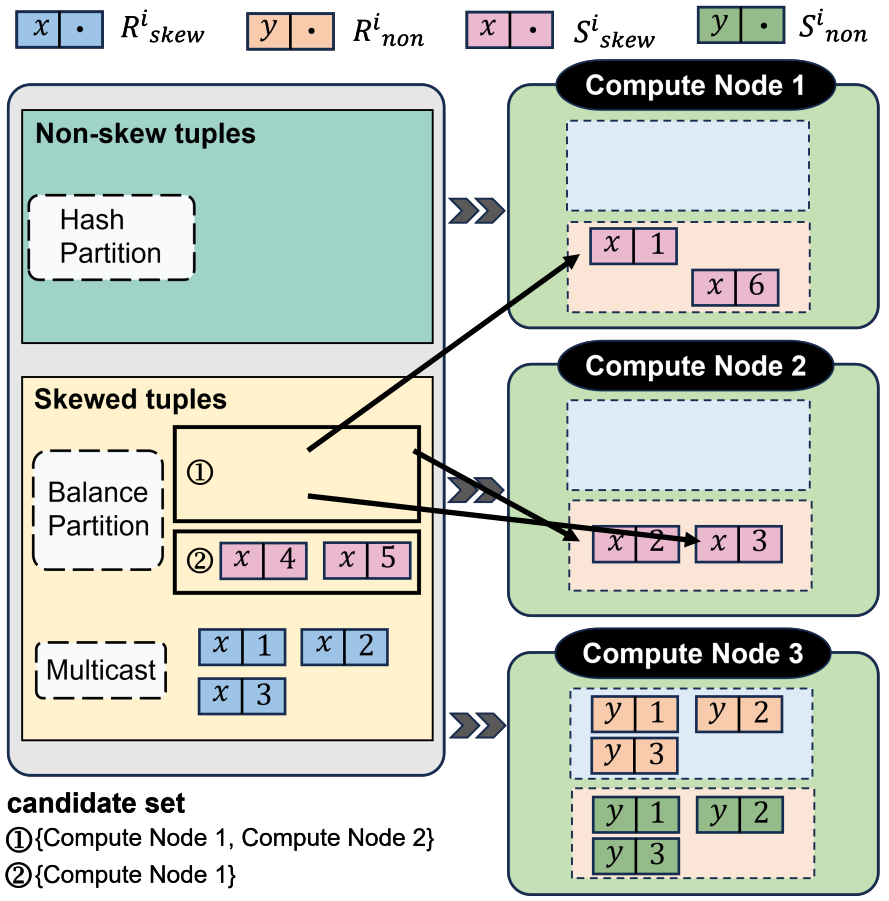


Redistributing partitions to nodes

Our schema: Bala-Join

BPPR(Balanced Partition and Partial Replication)

- Divide the probe side into **non-skewed** and **skewed** tuples *(skewed keys are **detected online** as tuples stream in)*
- Non-skewed tuples follow **standard hash partitioning**
- Their matching build-side tuples are also **hash-partitioned**
- Skewed tuples are routed to a **candidate node set** $U(x)$
- The matching build-side tuples are then multicast **only to** $U(x)$, reducing unnecessary network overhead

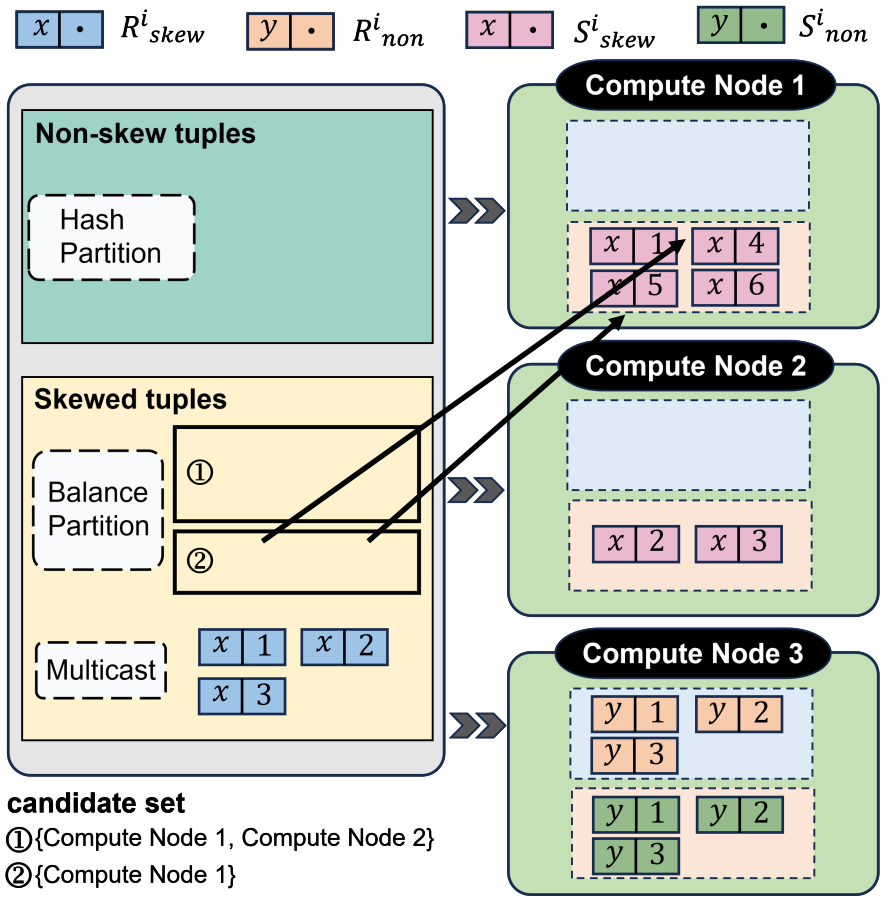


Redistributing partitions to nodes

Our schema: Bala-Join

BPPR(Balanced Partition and Partial Replication)

- Divide the probe side into **non-skewed** and **skewed** tuples *(skewed keys are **detected online** as tuples stream in)*
- Non-skewed tuples follow **standard hash partitioning**
- Their matching build-side tuples are also **hash-partitioned**
- Skewed tuples are routed to a **candidate node set** $U(x)$
- The matching build-side tuples are then multicast **only to** $U(x)$, reducing unnecessary network overhead

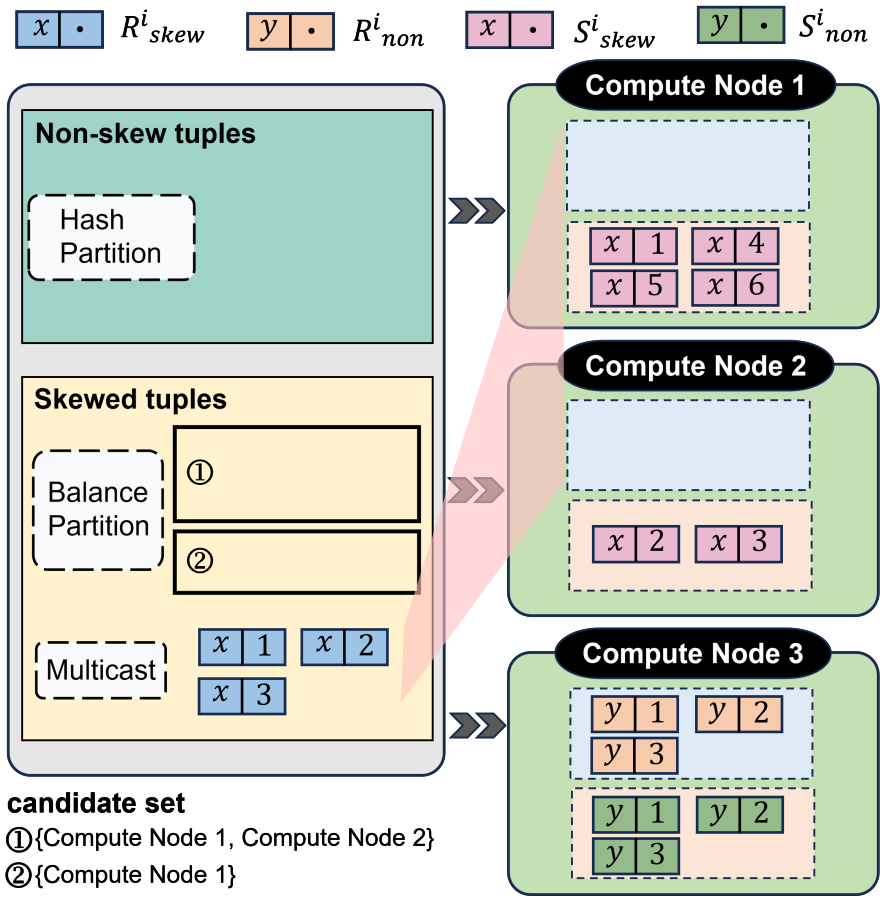


Redistributing partitions to nodes

Our schema: Bala-Join

BPPR(Balanced Partition and Partial Replication)

- Divide the probe side into **non-skewed** and **skewed** tuples *(skewed keys are **detected online** as tuples stream in)*
- Non-skewed tuples follow **standard hash partitioning**
- Their matching build-side tuples are also **hash-partitioned**
- Skewed tuples are routed to a **candidate node set** $U(x)$
- The matching build-side tuples are then multicast **only to** $U(x)$, reducing unnecessary network overhead



Redistributing partitions to nodes

Our schema: Bala-Join

Balanced Partition

- For each skewed key, BPPR maintains an **expandable candidate node set** $U(x)$
- New skewed tuples are assigned to nodes in $U(x)$
- The assignment tries to keep the balance factor within a threshold ε

$$B = (\max_j |\widehat{S}^j_{skew}| - \min_j |\widehat{S}^j_{skew}|)/\max_j |\widehat{S}^j_{skew}|$$

- If the current set cannot maintain balance, $U(x)$ **is expanded incrementally**

Result: Cluster-wide workload is balanced while the multicast scope remains as small as possible.

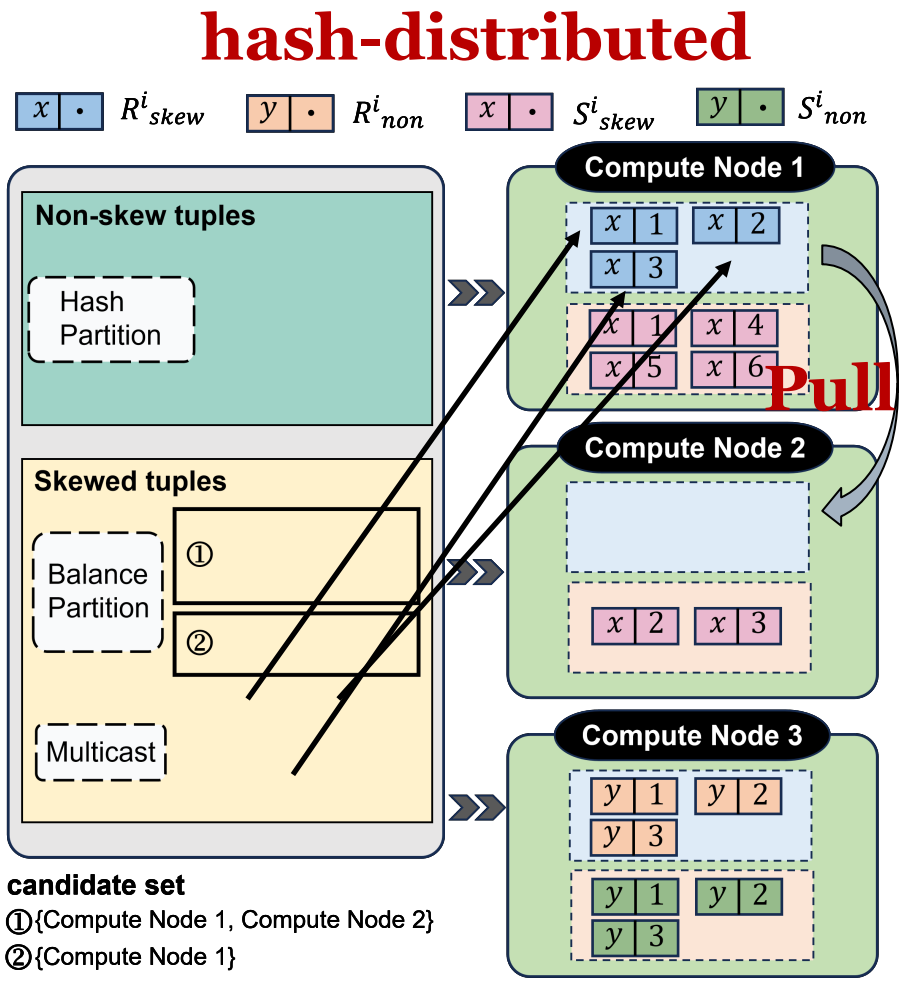
Our schema: Distributed Skew Detector and ASAP

Distributed Skew Detector

- Each data node uses a **local online detector** to identify skewed join keys in its **local probe stream**.
- This avoids expensive global consensus and enables **one-pass detection + redistribution**.
- Skewed probe tuples are passed to **BPPR** for adaptive routing.

ASAP

- ASAP provides **active signaling** and **asynchronous pulling** for build-side synchronization.
- Once a compute node receives skewed probe tuples, it is notified and can pull the **matching build tuples** from the corresponding hash node on demand.
- This preserves correctness without pre-scanning or materializing intermediate results.



Experiments: Setting

Dataset

- Synthetic dataset with Zipf distribution
- SSB-skew benchmark, scale factor = 10

Metrics

- Throughput
- Network traffic

Compared Solutions

Redistribution strategies

- GraHJ: standard hash partitioning
- PRPD: keeps skewed tuples local
- PnR: evenly distributes skewed tuples
- SFR: symmetric fragment replication
- **BPPR:** our redistribution strategy

End-to-end solutions

- Flow-Join: a comprehensive skew-aware Dist-HJ solution
- **Bala-Join:** our full solution

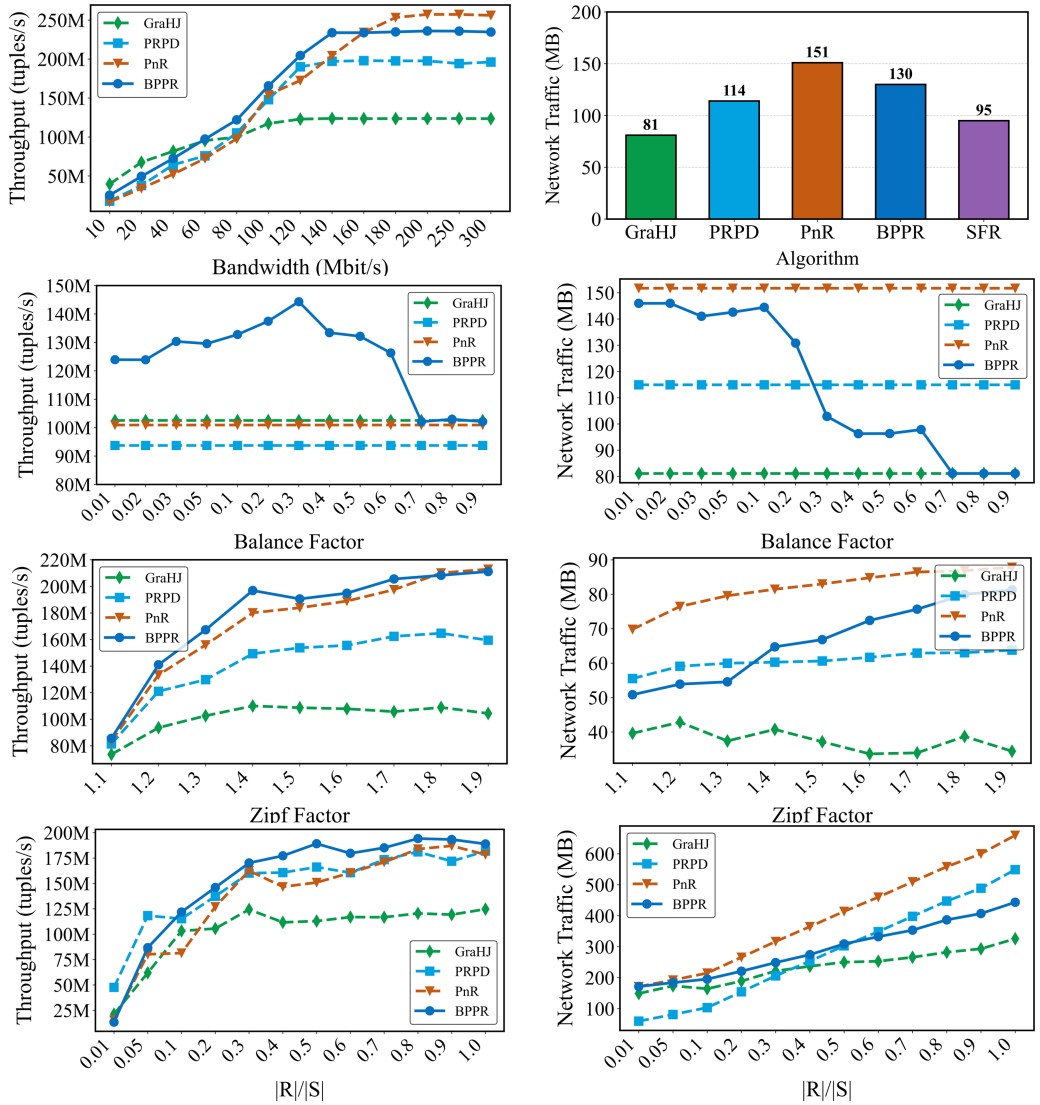
Experiments: Setting

- Cluster: Geo-distributed shared-nothing database over WAN
  - We emulate a CockroachDB-style shared-nothing database on **Huawei Cloud**.
  - The WAN deployment spans **three regions**: Beijing, Shanghai, and Guiyang.
  - We use **1–8 cloud servers** in each region, forming a cluster of **3–24 nodes**.
  - This layout reflects a typical geo-distributed configuration used in **Inspur's production business platform**.

Experiments: BPPR is more balanced

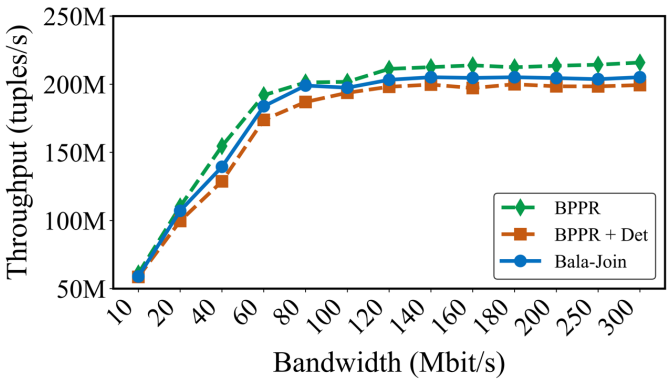
- Evaluate redistribution strategies directly under known skew information.
  - BPPR achieves the best overall adaptability across bandwidth, balance factor, Zipf factor, and $\frac{|R|}{|S|}$.
  - BPPR ranks 1st in average throughput across all four groups of experiments.
  - It also maintains competitive network overhead, achieving a better communication–computation trade-off.

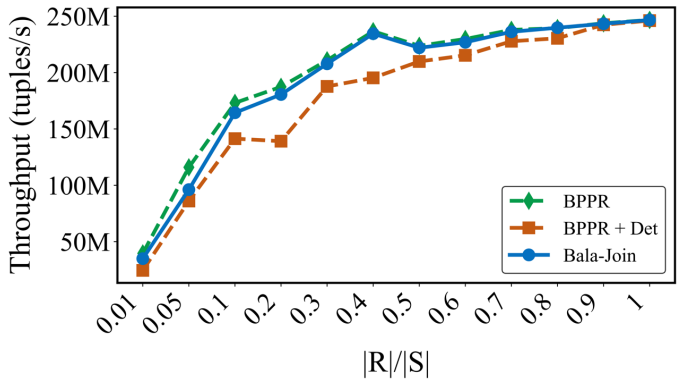
	Bandwidth	Balance Factor	Zipf Factor	$\|R\|/\|S\|$	Comprehensive Ranking
BPPR	**165M** (1)	**125M** (1)	**178M**, (1)	**154M** (1)	**622M** (1)
PnR	159M (2)	101M (3)	172M (2)	137M (4)	569M (2)
PRPD	140M (3)	94M (4)	142M (3)	148M (2)	524M (3)
SFR	127M (4)	81M (5)	131M (4)	140M (3)	470M (4)
GraHJ	105M (5)	103M (2)	102M (5)	103M (5)	413M (5)

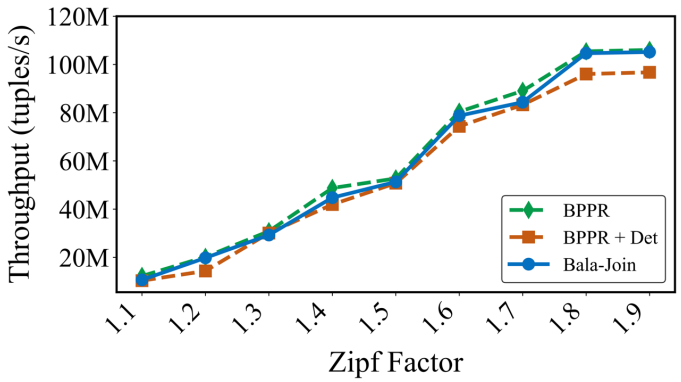


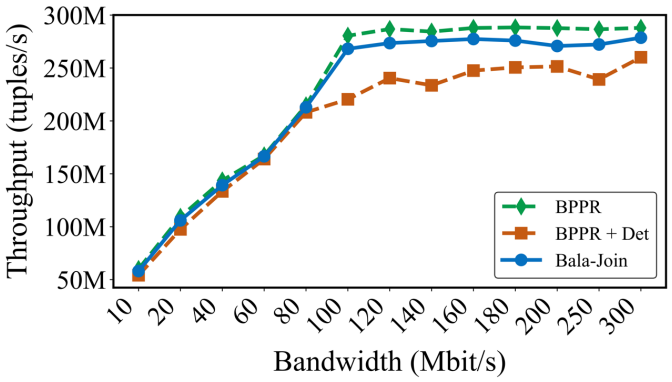
Experiments: Bala-Join is practical

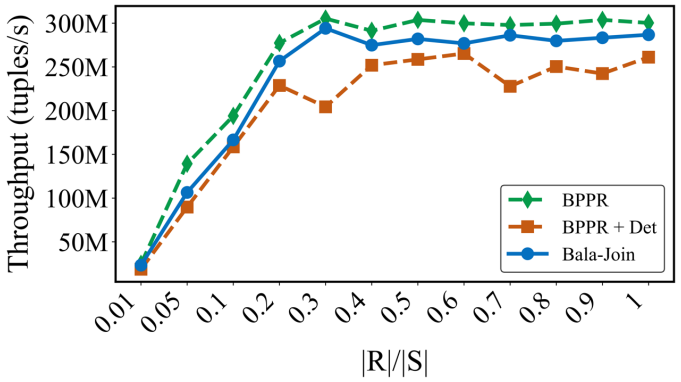
- The integrated detector introduces only moderate overhead.
- The detector adds about 5% overhead compared with ideal BPPR, while clearly outperforming BPPR+Det.

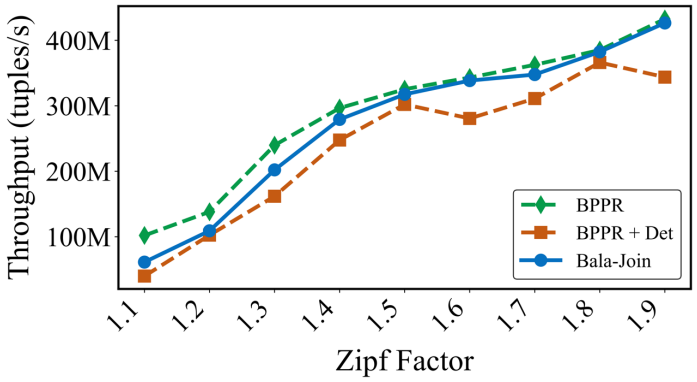






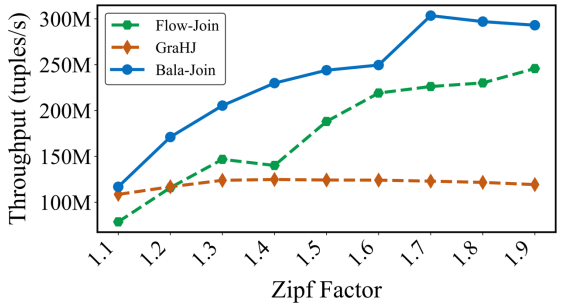


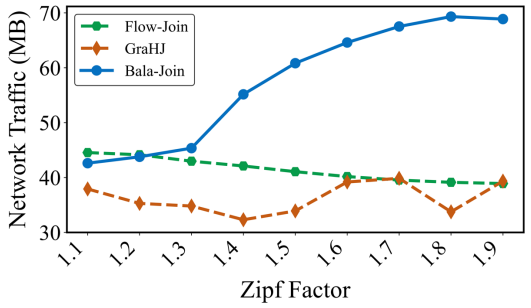


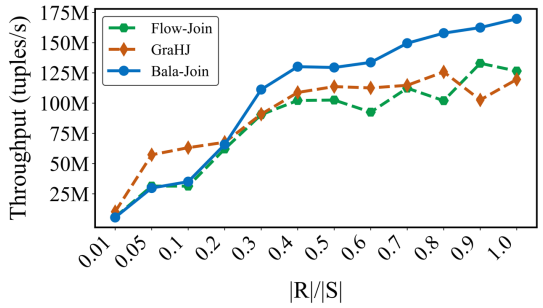


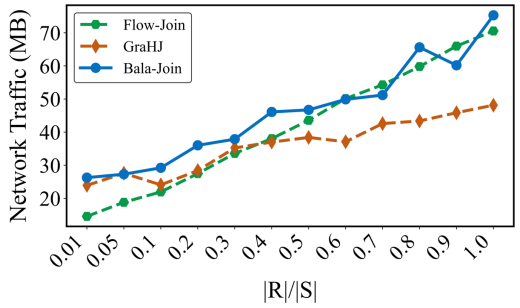
Experiments: Bala-Join is better end-to-end

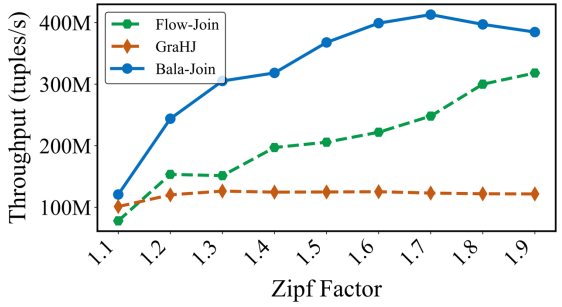
- Bala-Join evaluates the full Dist-HJ pipeline: detection, redistribution, and computation.
- It consistently outperforms Flow-Join and GraHJ in throughput.
- It achieves 25%–61% throughput improvement over Flow-Join.

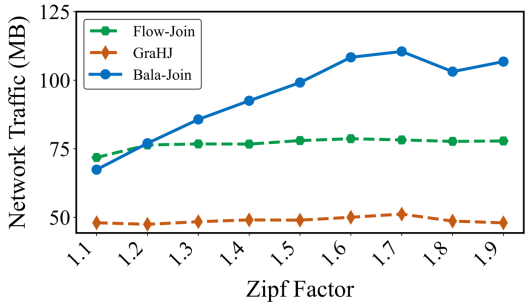


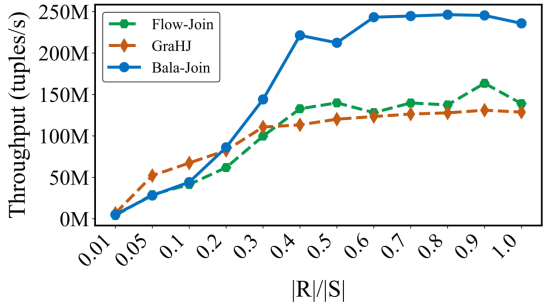


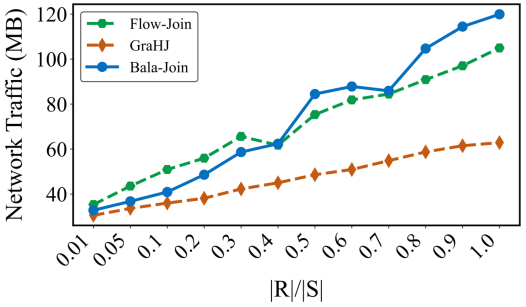












Outline

- Introduction
- Preliminaries
- Our schema
- Conclusions

Conclusion

- Bala-Join is an adaptive skew-aware Dist-HJ solution
- BPPR balances skewed redistribution with limited multicast scope
- The distributed detector and ASAP make the solution online and practical
- End-to-end throughput improves by 25%–61%

Thanks for attention!